5.1. Overlay Networks: Routing Overlays

An overlay network is a virtual or logical network that is created on top of an existing physical network. The internet, which connects many nodes via circuit switching, is an example of an overlay network.

An overlay network is any virtual layer on top of physical network infrastructure. This may be as simple as a virtual local area network (VLAN) but typically refers to more complex virtual layers from software-defined networking (SDN) or a software-defined wide area network (SD-WAN).

The overlay creates a new layer where traffic can be programmatically directed through new virtual network routes or paths instead of requiring physical links. Overlays enable administrators to define and manage traffic flows, irrespective of the underlying physical infrastructure.

SDN enables more flexible virtual networking that enables a more hands-off approach without changes to the physical underlay. SDN is an example of distributed computing where the actual processing is spread across multiple nodes, a departure from client-server computing where those routes were hardcoded.

5.1.1.Overlay network structure and protocols

Overlay network protocols include Virtual Extensible LAN (VXLAN), Generic Routing Encapsulation, Network Virtualization using GRE, Stateless Transport Tunneling and Network Virtualization Overlays.

Most network overlays work at Layer 3 in the Open Systems Interconnection (OSI) model, handling all traffic through the IP address. But, if a VLAN is created as an overlay, then the overlay would be done at Layer 2 with media access control (MAC) addresses.

In the case of SDN, the most common protocol for communication is OpenFlow, an open standard protocol that provides interoperability and is used in some fashion by most SDN.

Advantages of overlay networks

Network overlays provide some key benefits to networking, including the following:

- **Flexibility.** The overlay provides a more flexible networking approach by removing the hardcoded constraints of a physical network, which enables configuration tied to usage or function.
- **Management.** Overlays offer better access management by segmenting and joining devices logically instead of managing these components physically.
- **Security.** Overlay networks enhance security by segmenting traffic and restricting access by groups, individuals or devices. In the case of a network compromise -- when using SDN as an overlay -- an attacker's traffic can be detected and stopped more easily.
- **Redundancy and efficiency.** With an overlay, traffic has an easier time changing routes based on either traffic saturation or network interruptions.

Disadvantages of overlay networks

Despite the advantages of overlay networks, organizations should heed the potential challenges or disadvantages as well, including the following:

- **Extra layers of management.** IT would have to manage two different network layers daily. Most importantly, the layers must be managed in unison as the topology that the overlay expects needs to be accurately represented in the underlay.
- **Troubleshooting.** Again, this must occur for both the underlay and overlay.
- **Potential security exposure.** The negative effects of misconfiguration can be amplified across a wider set of devices or users.

Examples and uses of overlay networks

Some examples of overlay network deployments include virtual private networks, peer-to-peer networks, content delivery networks, voice over IP services and non-native software-defined networks. Other examples and uses of overlay networks are the following:

- **VLAN or VXLAN.** These networks are created at Layer 2 or encapsulated with Layer 2 to create logical segments for routing traffic.
- **Hypervisor and virtual servers.** Virtual networking creates virtual switches and virtual network cards that create an overlay for communicating between virtual machines or between the hypervisor and the rest of the network.
- **SD-WAN.** SD-WAN creates an overlay that manages a communication tunnel between two networks so that all the communications do not need to be hardcoded to the connection.
- **SDN.** SDN uses protocols like OpenFlow to create a virtual overlay that sits on top of network switches, enabling the switches to handle more of the data routing functions, optimizing data flow.

Overlay vs. underlay networks: What are the differences?

An overlay network is a network that is built on top of another network and is supported by its infrastructure. An overlay network decouples network services from the underlying infrastructure by encapsulating one network packet inside of another packet. After the encapsulated packet has been forwarded to the endpoint, it is de-encapsulated.

What is an underlay network?

An underlay network comprises the physical switches, routers and other devices that connect nodes and route data among them. An underlay network employs some physical network medium -- such as copper wire, fiber optic or even wireless -- for the physical transfer of data.

Every overlay requires an underlay to operate. In comparison to vehicle traffic on roads, the overlay is the traffic signs, lights and markings that direct traffic, and the underlay is the physical

street. One could change the direction of the traffic by changing the signage, while the actual road surface remains untouched.

5.1.2. Overlay networks and SDN

SDN is a quickly growing network strategy where the network operating system separates the data plane (packet handling) from the control plane (the network topology and routing rules). SDN acts as an overlay, running on the distributed switches, determining how packets are handled, instead of a centralized router handling those tasks.

From its inception, the Internet has adopted a clean model, in which the routers inside the network are responsible for forwarding packets from source to destination, and application programs run on the hosts connected to the edges of the network. The client/server paradigm illustrated by the applications discussed in the first two sections of this chapter certainly adhere to this model.

In the last few years, however, the distinction between *packet forwarding* and *application processing* has become less clear. New applications are being distributed across the Internet, and in many cases these applications make their own forwarding decisions. These new hybrid applications can sometimes be implemented by extending traditional routers and switches to support a modest amount of application-specific processing. For example, so-called *level-7 switches* sit in front of server clusters and forward HTTP requests to a specific server based on the requested URL. However, *overlay networks* are quickly emerging as the mechanism of hoice for introducing new functionality into the Internet

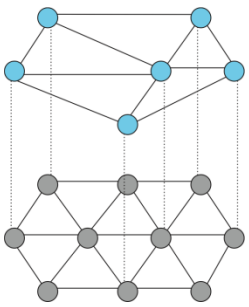


Figure 1. *Overlay network layered on top of a physical network.*

You can think of an overlay as a logical network implemented on top of some underlying network. By this definition, the Internet started out as an overlay network on top of the links provided by the old telephone network. Figure 1 depicts an overlay implemented on top of an underlying network. Each node in the overlay also exists in the underlying network; it processes and forwards packets in an application-specific way. The links that connect the overlay nodes are implemented as tunnels through the underlying network. Multiple overlay networks can exist on top of the same underlying network—each implementing its own application-specific behavior—and overlays can be nested, one on top of another. For example, all of the example overlay networks discussed in this section treat today's Internet as the underlying network.

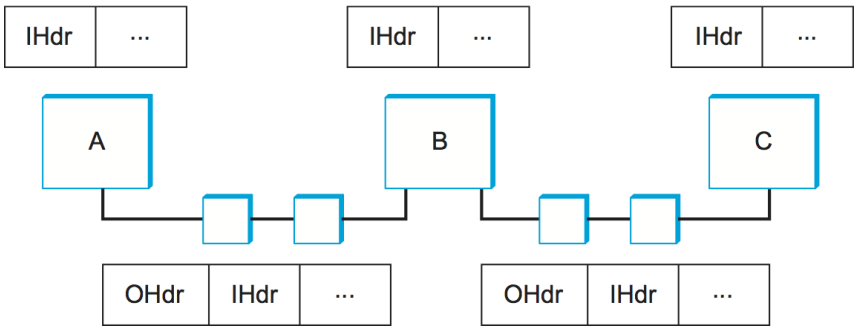


Figure 2. *Overlay nodes tunnel through physical nodes.*

We have already seen examples of tunneling, for example, to implement virtual private networks (VPNs). As a brief refresher, the nodes on either end of a tunnel treat the multi-hop path between them as a single logical link, the nodes that are tunneled through forward packets based on the outer header, never aware that the end nodes have attached an inner header. Figure 2 shows three overlay nodes (A, B, and C) connected by a pair of tunnels. In this example, overlay node B might make a forwarding decision for packets from A to C based on the inner header and then attach an outer header that identifies C as the destination in the underlying network. Nodes A, B, and C are able to interpret both the inner and outer header, whereas the intermediate routers understand only the outer header. Similarly, A, B, and C have addresses in both the overlay network and the underlying network, but they are not necessarily the same; for example, their underlying address might be a 32-bit IP address, while their overlay address might be an experimental 128-bit address. In fact, the overlay need not use conventional addresses at all but may route based on URLs, domain names, an XML query, or even the content of the packet.

5.1.3 Routing Overlays

The simplest kind of overlay is one that exists purely to support an alternative routing strategy; no additional application-level processing is performed at the overlay nodes. You can view a virtual private network (VPN) as an example of a routing overlay, but one that doesn't so much define an alternative strategy or algorithm as it does alternative routing table entries to be processed by the standard IP forwarding algorithm. In this particular case, the overlay is said to use "IP tunnels," and the ability to utilize these VPNs is supported in many commercial routers.

Suppose, however, you wanted to use a routing algorithm that commercial router vendors were not willing to include in their products. How would you go about doing it? You could simply run your algorithm on a collection of end hosts, and tunnel through the Internet routers. These hosts would behave like routers in the overlay network: As hosts they are probably connected to the Internet by only one physical link, but as a node in the overlay they would be connected to multiple neighbors via tunnels.

Since overlays, almost by definition, are a way to introduce new technologies independent of the standardization process, there are no standard overlays we can point to as examples. Instead, we illustrate the general idea of routing overlays by describing several experimental systems that have been built by network researchers.

5.2. Resilient Overlay Networks

Introduction.

A Resilient Overlay Network (RON) is an architecture that allows distributed Internet applications to detect and recover from path outages and periods of degraded performance within seconds. It is an application layer overlay on top of existing Internet routing substrate. The RON nodes monitor the functionality and quality of the Internet paths among themselves and use this information to decide whether to route packets directly over the Internet or by way of other RON nodes, optimizing application specific metrics.

eg. IP itself is an overlay network.

Purpose

Add functionalities to IP for eg. Multicast, resilience or fault tolerance and resilience service composition. The motivation behind are

Multicast

- In the current proposals there are no congestion control mechanisms
- there is no accounting mechanism for the bandwidth one is carrying

Resilience

The routing BGP has many drawbacks. It is a path vector protocol and converges slowly as opposed to a link state protocol. The only metrics used by BGP is the *path length*. Also BGP hides underlying topological details in the interests of scalability and policy enforcements. As a result the recovery mechanisms take minutes to converge. For eg even to implement policy decision BGP uses path length. Consider a topology where there are four nodes a,b,c,d. There are links between c--a, c--d, a--d, d--b, a--b. Suppose the policy by "a" is not to give transit service to "c" unless the c--d--b link is also down. For this "a" advertises to "b" a path to "c" via "a--a--a--a--a" instead of via "a" only. This way "a" abuses the path length to implement policy.

The BGP availability is also very small. Only 10% of the paths is available for more than 95% and on the other hand a telephone line is available for 99.999%.

One thing to note is that multi homing is not a solution because

- It only increases the number of options but does not increase availability as there is no improvement in convergence
- Also it is not good for address aggregation

RON Model

The RON model will have a set of *designated RON nodes*. Those will communicate among themselves through a *fully mesh network* and exchange performance and reachability information with a *link state* based dissemination and do routing based on that. These will give us the following advantages

1. fast recovery from failures
2. application specific metrics
3. packet classification policies

Besides that there should be some metrics and the architecture should store performance based on those metrices.

Usage Models

- a specific application constructs and uses a RON
- A network administrator constructs an overlay
- Overlay ISP

In overlay ISP there are several ISP's that form a RON and RON can be brought from them.

Failure Detection in RON

- send UDP heartbeats

Usually these heartbeats are small and do not cause much overhead so we can send faster heartbeats under some constraint. But the failure detection is application specific. For eg. the criterion for failure detection for an FTP application will be different than that for a video conferencing application.

But high rate of heartbeats is not always good because of false alarms. Incase of too fast heartbeats there can be flip flops leading to packet reordering (particularly bad for video conf. because there are some special frames which contain large amount of information)

Metrics

- Latency

It is calculated by the RTT measurement of the heartbeats. We need to know how stable the RTT values are. Study has shown that these RTT's are stable over 15-minutes to 1 hour. Usually EWMA of the RTT is taken therefore random spikes don't matter and results remain stable for 15min to 1 hour period. The total RTT will be the summation over individual links.

- Packet loss rate

Packet loss rate is calculated by the heartbeats. The equation used is 1-(1-p1)(1-p2) for two links where p1 and p2 are packet loss rates in link1 and link2.

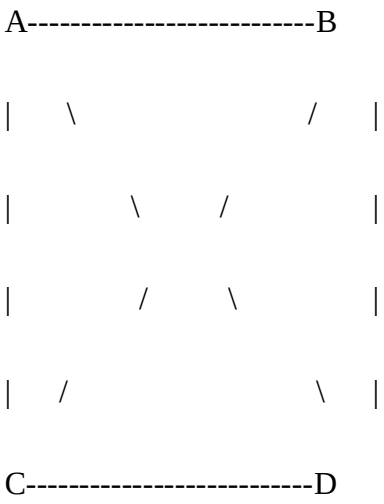
- bandwidth

The bandwidth can be measured by sending back to back packets. In TCP the receiving end will receive packets at the rate of the bottleneck bandwidth. But even in that there are two cases

1. If the intermediate routers implement FIFO then the above is not true
2. If the intermediate routers implement WFQ then the above is true

The bandwidth is very unstable. Usually the variation is about a factor of 2. Therefore we use alternate paths iff alternate bandwidth > 2 times current bandwidth otherwise there will be a lot of switching.

Also only two hop alternate paths are considered. Consider the topology below



There are links A-B, A-C , A-D, B-C, B-D, C-D.

If A-B goes down then using the above rule we will use either A-D-B or A-C-B only.

As a consequence of all the above in case of Mobile IP it is not always necessary that the triangle routing will cause greater latency than a direct route.

5.3. Peer-Peer Networks – Bit Torrent – Distributed Hash Tables

5.3.1. Peer-Peer Networks

Although peer-to-peer networking has received a lot of attention recently due to the ongoing battle with the music and movie industry, it is not a new concept. In its simplest definition, peer-to-peer is described as :

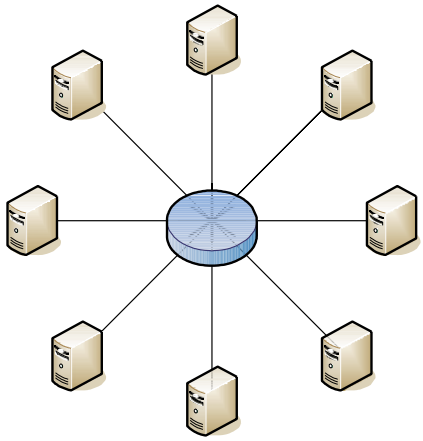


Figure 1

This means that conceptually, peer-to-peer computing is an alternative to the traditional client / server architecture where there typically is a single (or small cluster) server and many clients (figure 1).

Sticking with the previous definition, the concept of peer- to-peer can be traced back to the 1960's when the early implementation of the Internet (ARPANET) was a peer- to-peer network in which all its peers were equals. However, this definition can be fitted into many scenarios. The Domain Name System (DNS) is a good example of a blend between the traditional peer-to-peer networking and a hierarchical model of information ownership. A more precise definition is stated in as:

"A distributed network architecture may be called a Peer-to-Peer (P-to-P, P2P,...) network, if the participants share a part of their own (hardware) resources (processing power, storage capacity, network link capacity, printers,...). These shared resources are necessary to provide the Service and content offered by the network (e.g. file sharing or shared

workspaces for collaboration). They are accessible by other peers directly, without passing intermediary entities. The participants of such a network are thus resource (Service and content) providers as well as resource (Service and content) requestors(Servant-concept)."

As there exists many forms of peer-to-peer networks, both with and without some with a form of central entity, this definition was further refined in. This refinement introduced a classification of peer-to-peer networks as either pure (Figure 2) or hybrid (Figure 3). This was done so one is be able to distinguish between peer-to-peer networks with or without the previously mentioned central entity.

Peer-to-peer network topologies

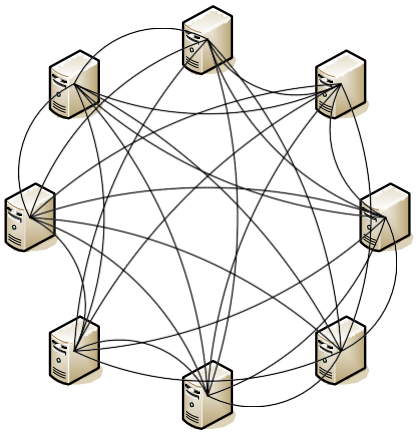


Figure 2

The "pure" peer-to-peer concept is a network in which the peers themselves are the only entities allowed within. Or as stated in :

"A distributed network architecture has to be classified as a "Pure" Peer-to-Peer network, if it is firstly a Peer-to- Peer network according to Definition 1 and secondly if any single, arbitrary chosen Terminal Entity can be removed from the network without having the network suffering any loss of network service."

This is concept is showed in Figure 2. Here we can clearly see that only what seem to be equal peers are present in the network. All are interconnected which means that any peer can be removed, without this having any fatal consequences on the network, i.e. there is no single point of failure.

Hybrid peer-to-peer

Different from a "pure" peer-to-peer concept, where only the peers themselves were allowed in the network, a "hybrid" network will always include some sort of central entity. This is shown

in Figure 3.In [17] the hybrid peer-to-peer concept is defined as:

> "A distributed network architecture has to be classified as a "Hybrid" Peer-to-Peer network, if it is firstly a Peer-to-Peer network according to Definition 1 and secondly a central entity is necessary to provide parts of the offered network services."

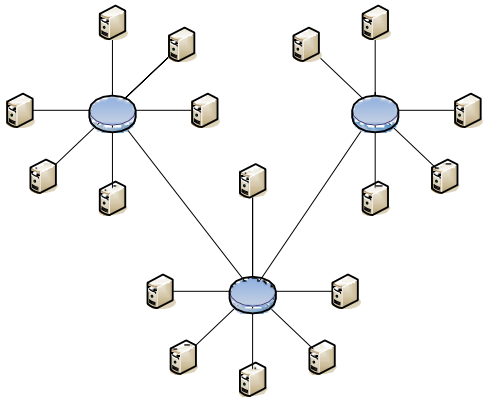


Figure 3

As Figure 3 shows, there are now hub nodes in the network which connects different networks. If one of these nodes go down, e.g. the center hub node of Figure 3, some parts of the network are suddenly separated. This makes the Hybrid peer-to-peer network more vulnerable to attacks or failure.

5.3.2. BitTorrent

BitTorrent is a technology/protocol which makes the distribution of files, especially large files, easier and less bandwidth consuming for the publisher. This is accomplished by utilizing the upload capacity of the peers that are downloading a file. A considerably increase in downloaders will only result in a modest increase in the load on the publisher hosting the file.

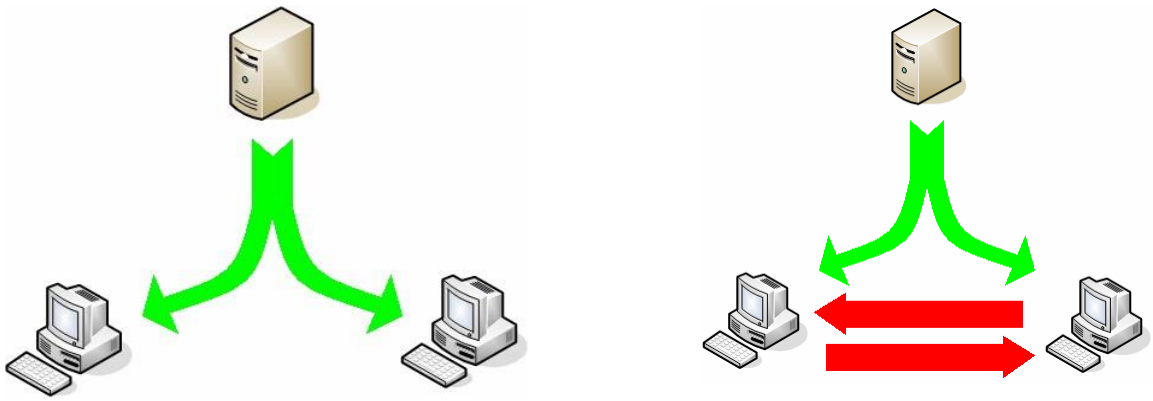


Figure 4 – The basic flow of the BitTorrent protocol.

The illustration in Figure 4 shows the basic flow of BitTorrent. The figure on the left shows a client-server approach to download. The peers download from the server simultaneous. If we assume the upload capacity of the server is the same as the download capacity of a peer, the time for the download to finish will be two times the time if only one peer where downloading from the server. The figure on the right shows an approach similar to BitTorrent. By splitting the file and send one part to each peer, and let the peers download the part they are missing from each other, both download time and load on the server is reduced. Of course, the BitTorrent protocol is much more sophisticated than this simple example, but this shows the basic idea.

BitTorrent is completely free and open-source. This is doubtless an accomplice reason for the big success. But the way BitTorrent works also differs considerable from the other peer-to-peer protocols out there.

The problem with many “traditional” peer-to-peer file sharing protocols, in the eyes of Bram Cohen, is that most users have different speeds downlink and uplink. This means that even though a user has plenty of bandwidth downlink, the speed of a file transfer to him will be

restricted by a much smaller bandwidth uplink from the user he downloads from [20]. Let us take a simple example: assume that a user wants to fetch a file from another user. They both have the same connection; 2,5Mbit/s downlink and 600kbit/s uplink[2]. What determines the speed of the transfer between the two is thus the uplink of 600kbit/s (i.e. the bottleneck), and the capacity of the downlink is unessential. Most people have an uplink below 1Mbit/s, and sometimes as little as about 10% of the downlink bandwidth. Sure, recently we have seen SHDSL and similar, but the main majority of users have asymmetric rates and this is highly likely to continue in the near future.

Areas of usage

Piracy and illegal distribution is the first thing brought to mind when you hear about peer-to-peer file sharing. Not unreasonable, since the majority of traffic is illegal exchange of copyrighted material. But the BitTorrent protocol has features which makes it usable for totally legitimate purposes.

It is already possible to download Linux distributions using BitTorrent which enables much faster download than the regular FTP or HTTP can provide. As mentioned earlier, it was also for this purpose Bram Cohen presented his protocol. The internet browser Opera can also be downloaded using BitTorrent.In addition to increased download speed, it has the benefit that it eases the pressure on the servers when a new version is published. When they released version 8 of their browser their servers went down due to a massive demand .Legal usage is growing and many use the protocol to distribute their home-made music or short films, game demos and other legal content .

BitTorrent also has potential business usage. Distribution of ISO-images, operating systems, large software and patches can be done at higher speeds using BitTorrent .How often haven't you waited too long for the newest security update from Windows to be finished

2 The BitTorrent architecture

"I call architecture frozen music…"

- Goethe

This section will explain the standard BitTorrent architecture as described in [4]. Later a new

pure peer-to-peer approach will be explained.

The BitTorrent architecture normally consists of the following entities [27]:

- a static metainfo file (a “torrent file”)
- a ‘tracker’
- an original downloader (“seed”)
- the end user downloader (“leecher”)

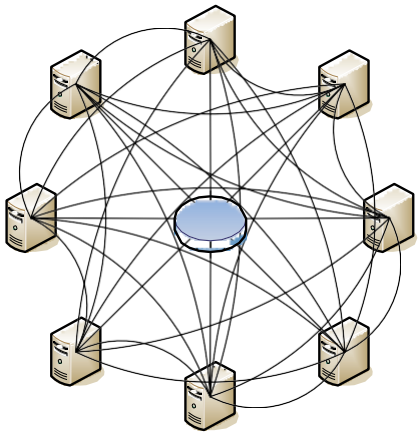


Figure 5 – BitTorrent in its original form matches the “hybrid” peer-to- peer concept. It’s all about the torrent file, the centralized tracker and the associated swarm of peers. The centralized tracker provides the different entities with an address list over the available peers. These peers will then contact each other to download pieces of the file from eachother.

The first step in publishing a file using BitTorrent is to create a metainfo file from the file that you want to publish. The metainfo file is called a “torrent”. The torrent file contains the filename, size, hashing information and the URL of the “tracker”. The “torrent” is needed by anyone who wants to download the file the torrent is created from. The torrent file can be distributed by e-mail, IRC, http etc.

The torrent is created by using a free program. This functionality is also commonly included in the BitTorrent clients. To download or “seed” a file, you need a BitTorrent client. The BitTorrent client is a free application that administrates the download procedure. There are several different BitTorrent clients available [28]. They all support the standard BitTorrent protocol, but may differ and be incompatible with each other regarding certain features [29]. A BitTorrent download is started by opening the torrent file in the BitTorrent client.

The tracker keeps a log of peers that are currently downloading a file, and helps them find each other. The tracker is not directly involved in the transfer of data and
does not have a copy of the file. The tracker and the downloading users exchange information using a simple protocol on top of HTTP. First, the user gives information to the tracker about which file it’s downloading, ports it’s listening on etc. The responce from the tracker is a list of other users which are downloading the same file and information on how to contact them. This group of peers that all share the same torrent represents a ‘swarm’.

However, making a torrent file is not enough to make the file you want to distribute available. An original downloader known as a “seed” has to be started. A “seed” is a user that has the entire file. A downloading user that has nothing or only parts of a file is called a “leecher”. The “seed” must upload at least one complete copy of the file. Once an entire copy is distributed amongst the other downloaders, the ‘seed’ can stop uploading and the download will still continue for all downloaders as long as there are enough people downloading the file, and all pieces of the file are available. For a popular file one complete copy from the seed

may be enough while for a less popular file, continues uploading by the seed may be necessary. The result is that the bandwidth requirements for the publisher are less if many people are downloading. This is the complete opposite of what would happen if any server-client solution where to be used. The bandwidth requirement of the tracker is also very low even though the number of downloaders is high.

When creating the torrent file from the original file, the original file is cut into smaller pieces, usually 512 KB or 256Kb in size. The SHA-1 hash codes of the pieces are included in the torrent file. The downloaded data are verified by computing the SHA-1 hash code and comparing it to the SHA-1 code of the corresponding piece in the torrent file. This way the data is checked for errors and it guarantees to the users that they are downloading the real thing. Whenever a piece is downloaded and verified, the downloading peer reports to the other peers in the swarm about its new piece. This piece is now available for other peers. We will now go deeper into piece selection and other details of the protocol.

Algorithms

In many other peer-to-peer file sharing protocols the swapping of files happens one-to-one, meaning that the user himself chooses another peer to download from. As explained earlier, the concept of BitTorrent is to be able to download from many other peers simultaneously. This requires a way of knowing which peers to download what pieces of the file from, with the goal of receiving the complete file as quickly as possible. This needs to happen without the involvement of the end user, and thus needs to be self-configuring.

Choosing peers to connect to is a two-sided problem. First, we need a way of finding the best sequence of downloading the pieces. This is determined by the piece selection algorithm. But this is not enough. A peer, who has the piece you want, might not let you download it. Strategies for peers not allowing other peers to download from them is known as choking, and concerns resource allocation. These two concepts are examined in the following pages.

The Piece Selection Algorithm

How BitTorrent selects what pieces of the file to download have great impacts on the performance of the protocol. It is important to be smart when selecting pieces, in order to not end up a situation where every peer has all the pieces that are currently available and none of the missing ones.

The goal is to replicate different pieces on different peers as soon as possible. This will increase the download speed, and also make sure that all pieces of a file is somewhere in the network if the seeder leaves. We will go through several policies which combined make up the piece selection algorithm.

Sub-pieces

BitTorrent uses TCP and it is thus crucial to always transfer data or else the transfer rate will drop because of the slow start mechanism. The pieces are further broken into sub-pieces, often about 16kb in size. The protocol makes sure to always have some number of requests (typically five) for a sub-piece pipelined at any time. When a new sub-piece is downloaded, a new request is sent. Sub-pieces can be downloaded from different peers.

Policy #1: Strict Policy

Once a sub-piece has been requested, the remaining sub-pieces for that particular piece are requested before sub-pieces from any other piece. This helps us getting a complete piece as quickly as possible. [4]

Policy #2: Rarest First

The main policy in BitTorrent is that of "rarest first". This means that when a peer selects the next piece to download, it selects the piece which the fewest of their peers have. There are several reasons for this being a good policy [4][12]:

- **Spreading the seed:** Rarest first makes sure that only "new" pieces are downloaded from the seed. In the beginning, the seed will be a bottleneck since it is the only one with any piece of the file. A downloader can see what pieces their peers have, and the "rarest first"-policy will result in that the pieces fetched from the seed are pieces which have not already been uploaded by others.
- **Increased download speed:** The more peers that have the piece, the faster the

download can happen, as it is possible to download sub-pieces from different places. We want to replicate rare pieces so they can be downloaded faster.

- **Enabling uploading:** A rare piece is most wanted by other peers, and by getting a rare piece others will be interested in uploading from you.
- **Most common last:** It is sensible to leave the most common pieces to the end of the download. Since many peers have it, the probability of being able to download them later is much larger than that of the rare pieces.
- **Prevent rarest piece missing:** When the seed is taken down, it is important that all the different pieces of the file are distributed somewhere among the remaining peers. By replicating the rarest pieces first, we reduce the risk of missing one or more pieces of a file when the seeder leaves.

Policy #3: Random First Piece

Once you start downloading, you don't have anything to upload. It is important to get the first piece as fast as possible, and this means that the "rarest first"-policy is not the most efficient. Rare pieces tend to be downloaded slower, because you can download it's sub-pieces from only one (or maybe a few) other peers. As mentioned earlier, multiple peers with the same piece increase the download speed. The policy is then to select the first piece randomly. When the first piece is complete, we change to "rarest first". [4]

Policy #4: Endgame mode

Sometimes a piece might be downloaded from a peer with a slow transfer rate. This can potentially delay the finishing of a download. To prevent this we have the "endgame mode". Remember the pipelining principle, which ensures that we always have a number of requests (for sub-pieces) pending, the number often being set to five. When all the sub-pieces a peer lacks are requested, this request is broadcasted to all peers. This helps us to get the last chunk of the file as quickly as possible. Once a sub-piece arrives, we send a cancel-message indicating that we have obtained it and the peers can disregard the request. Some bandwidth is of course wasted by this broadcasting, but in practice this is not very much because of the short periode of the endgame mode. Resource Allocation

No centralized resource allocation exists in BitTorrent. Every peer is responsible for maximizing its download rate. A peer will, naturally, try to download from whoever they can. To decide which peers to upload to, a peer uses a variant of the "tit-for-tat" algorithm. The "tit-for-tat"-strategy comes from repeated game theory, and is a strategy of cooperation based on reciprocity. The essence is to do onto others as they do onto you [11]:

1. On the first move cooperate.
2. On each succeeding move do what your opponent did the previous move.
3. Be prepared to forgive after carrying out just one act of retaliation (i.e. have a recovery mechanism to ensure eventual cooperation).

The Choking Algorithm

Choking is a temporary refusal to upload to another peer, but you can still download from him/her. To cooperate peers allow uploading, and to not cooperate they "choke" the connection to their peers. The principle is to upload to peers who have uploaded to you recently; i.e. "if you scratch my back, I'll scratch yours". The goal is to have several bidirectional connections at any time, and achieve "Pareto efficiency"[4] [4].

So, the big question is how to determine which peers to choke and which to unchoke. A peer always unchokes a fixed number of its peers (the default is four). Deciding which peers to un-choke is determined only by the current download rates. It has been chosen to use a 20-second average to decide this. Due to the usage of TCP it's not desirable to choke and un-choke too rapidly. Thus, this is calculated every ten seconds. [4]

The result is that any peer will upload to peers which provide the best download rate. The other way around; if your upload rate is high more peers will allow you to download from them. This means that you can get a higher download rate if you have many uploads. This is the most important feature the BitTorrent protocol. It prohibits a large number of "free riders" which are peers who only download and don't allow uploading. In order for a peer-to-peer-network to be efficient all peers have to contribute to the network. This restriction is not present in most other peer-to-peer protocols and applications, and is one of the reasons why BitTorrent has become so popular.

Optimistic unchoking

BitTorrent also allows an additional unchoked peer, where the download rate criterion isn't used. This is called optimistic unchoking. The reason for this is to see if there are any currently unused connections which might be better than the ones in use. Which peer is the optimistic unchoke is shifted every 30 seconds. This is considered to be enough time for the upload to boost up to full speed and for the download to start and obtain full speed. If this new connection turns out to be better than one of the existing unchoked connections, it will replace it [4]. This is quite similar to stage one of "tit-for-tat".

Anti-snubbing

What happens if a peer suddenly is choked by all peers it was downloading from? We then have to find new peers, but the optimistic unchoking mechanism only checks one unused connection every 30 seconds. To help the download rate recover more rapid, BitTorrent introduces "snubbing"[5]. If a client hasn't got anything from a particular peer for 60 seconds, it presumes that is has been "snubbed". Following the mentality of "tit-for-tat" it retaliates and refuses to upload to that peer (except if it becomes an optimistic unchoke). It will then increase the number of optimistic unchokes in order to try to find new connections quicker.

Upload only

We see that using the choking algorithm implemented in BitTorrent we favor peers which are kind to us. If I can download fast from them, they are allowed to upload from me. But what if I don't have any downloads? Then it's impossible to know which peers to unchoke using the presented choking algorithm. Thus, when a download is completed we use a new choking algorithm which unchokes the peers with the highest upload rate. This ensures that pieces get uploaded faster and that they get replicated faster. Peers with good upload rates are also probably not being served by others.

Concluding remarks

The BitTorrent protocol selects pieces by using the following four simple policies:

- **Policy #1: Strict Policy:** Until a piece is assembled, only download sub-pieces for that piece.
- **Policy #2: Rarest First:** Determine the pieces that are most rare among your peers and download those first.
- **Policy #3: Random First Piece:** Select a random piece of the file and download it.
- **Policy #4: Endgame mode**: When all the sub-pieces that a peer doesn't have are actively being requested, these are requested from every peer.

These policies ensure that the pieces are replicated, and that every peer has the largest probability of retrieving the complete file, as quickly as possible.

Inspired by "tit-for-tat", the choking algorithm tries to prohibit "free riders" from destroying the dynamics of the peer-to-peer network. If a peer blocks other peers from uploading, he will soon be choked by them and thus affecting his download rate. The choking algorithm correlates the download rate to the upload rate. Uploading is encouraged and the price in return is a better chance for faster download.

All this functionality is implemented inside the protocol. BitTorrent configures the connections to the peers without the involvement of the user, and is thus self-configuring. Decisions are made inside the client of every peer, based on the list of possible peers and the algorithms presented. The way the protocol configures these connections makes it very dynamic and it "achieves a higher level of robustness and resource utilization than any currently known cooperative technique." .

Important improvements

Bulk traffic marking

Version 4 of the protocol, released in March this year, had one technical improvement which made it more business-friendly. The BitTorrent traffic is now marked as bulk traffic which makes traffic shaping a lot easier .

Before this refinement, the large volume of BitTorrent traffic could easily have great negative

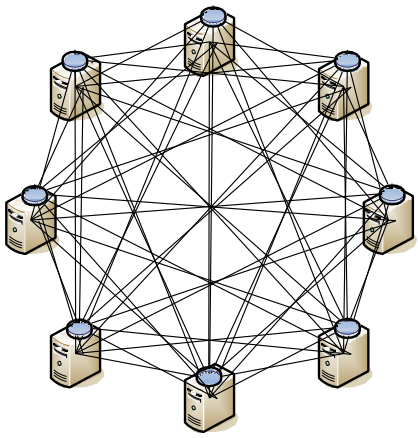
impacts on real-time traffic such as VoIP. The files transferred are often hundreds of megabytes and for a LAN with many users the total amount of traffic can be a major problem and slow down the network. When the BitTorrent file transfers are marked as bulk, almost any standard traffic shaping tool can be used to mange the network traffic.

Decentralized tracker

With one centralized tracker the BitTorrent network is not very fault tolerant. If the tracker goes down the file will no longer be available, since there are no way the peers could know about each other. One centralized tracker also makes the network vulnerable to denial of service attacks.

In May 2005 a version 4.1 was released by Cohen and the big newsflash was support for a decentralized tracker. In this new version (a beta release), all you have to do is to publish your .torrent-file on a webpage, blog etc, and no tracker specification is necessary. Use of a dedicated tracker is still supported, and the publisher can choose the preferred mode.

The tracker is distributed in the sense that every client or node in the network now acts a lightweight tracker. The solution is based on distributed hash tables (DHTs). This makes it possible to share files with minimal resources, but no guarantees can be made as respect to reliability.



In addition to making it easier for a more novice user to share his files through a simple website or a blog, one of the other driving forces for the trackerless mode was economy. A website owner might have to pay (to the web server owner) according to the traffic on his website. The more people who want to download your file, the larger the traffic the website generates the lager the bill will be for the owner. Cohen thinks this is unfair as compared to broadcast by a traditional TV-station: whether five people or five million people receive your broadcast, your cost is the same. By removing the tracker from the web server, the only traffic generated will be the fetching of the .torrent file. The web server (tracker) will no

Figure 6 – BitTorrent with a decentralized tracker structure

longer be involved in keeping track of where all the users are located, resulting in large bandwidth savings and near-broadcast economies.

The trackerless version is still in its childhood and the majority of BitTorrent file sharing will probably involve a traditional tracker for quite a while. The new solution is very promising and as it matures it will probably become increasingly popular. The next section will explain the fundamentals of DHTs which the distributed tracker is based on.

5.3.3. Distributed Hash Tables

A DHT is a decentralized distributed system [25]. The system consists of a set of participating nodes and a set of keys. The DHT performs the function of a hash table. Key and value pair can be stored and a value can be looked up if the correct key is provided. What separates a DHT from an ordinary hash table is that the storage and lookups are distributed among the nodes (machines) in the network. All nodes are peers that can join and leave the network freely. DHTs makes provable guarantees about performance despite the seemingly chaos created by random joining and leaving peers. Any DHT protocol emphasizes the following characteristics;

- **Decentralization**: the system is collectively created and maintained by the nodes without any central coordination.
- **Scalability**: the system performs efficiently even with thousands or millions of nodes.
- **Fault tolerance**: the system should be reliable even with nodes continuously joining, leaving or failing.

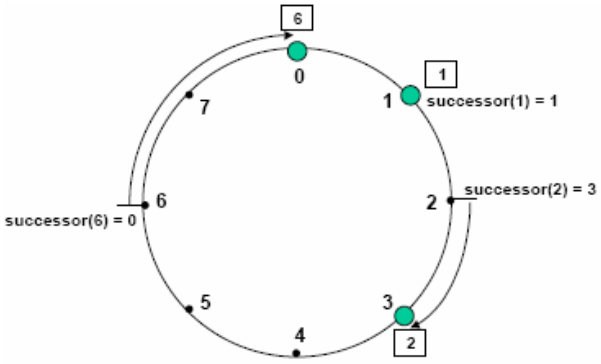
The DHT structure consists of two parts, the keyspace partitioning and the overlay network. The keyspace partitioning handles the partitioning of keys among the nodes in the network. The overlay network connects nodes and let them find the owner of a key.

There are several DHT protocols in use [25]. Chord, Pastry, Tapestry and Kademlia are a few of them. These protocols are more similar than they are different. “(..) the various available schemes all fit into a multidimensional matrix. Take one, make a few tweaks and you end up with one of the other ones” [24].

The Kademlia protocol can be seen as an evolved version of Chord. The Kademlia protocol is used in the Azureus and the BitTorrent client. There are also several other network that use these protocol. We will therefore use these two as examples of DHTs in the rest of this paper.

Keyspace partitioning

Each node in the network is assigned a single key which will be its identifier. To map keys to nodes DHT variants uses consistent hashing [9]. This type of hashing provides hash table functionality without the need to change the mapping of keys if a bucket is added or removed. In most traditional hash tables, a change in the number of buckets causes nearly all keys to be remapped. In consistent hashing the mapping of keys to buckets does not significantly change. This is essential considering the number of nodes who could at anytime be removed or added to the network. A change in ownership of keys would result in bandwidth intensive movement of objects in one node to another. By minimizing this reorganizing of keys, the network can support a high number of nodes entering and leaving the network.



The DHT protocols Chord [19] and Kademlia [10] uses the same type of consistent hashing. The scalability of consistent hashing is improved by avoiding the requirement that every node know about every other node.

In Chord and Kademlia each node is assigned a unique ID of *m* bits. Consistent hashing assigns keys to nodes as follows [19].

Figure 7 - Example of keyspace partitioning from[19].

The figure shows an identifier circle with *m* = 3. The circle has three nodes: 0, 1, and 3. The successor of identifier 1 is node 1, so key 1 would be located at node 1. Similarly, key 2 would be located at node 3, and key 6 at node 0.

If a node were to join with identifier 7, it would capture the key with identifier 6 from the node withidentifier 0.

Identifiers are ordered in an identifier circle modulo 2^m. Key *k* is assigned to the first node whose identifier is equal to or follows (the identifier of) *k* in the identifier space. This node is called the successor node of key *k*, denoted by successor(*k*). If identifiers are represented as a circle of numbers from 0 to 2^m - 1, then successor(*k*) is the first node clockwise from *k*. The keyspace is therefore split into contiguous segments. The endpoints are the node identifiers.

To maintain a consistent hash mapping when a node *n* joins the network, certain keys previously assigned to *n*'s successor now become assigned to *n*. When node leaves the network, all of its assigned keys are reassigned to *n*'s successor. No other changes in assignment of keys to nodes need occur.

Each node in the network is in it self a standard hash table. To access this table for storing or retrieving of data you need to find the right node in the network and then do normal store or lookup. To determine which node is the appropriate for a specific key the same approach as for finding the next node is used. Generate hash of *m* bits for the key you want to find. This hash is treated as a node ID and a search for a successor is started. The search can start at any point on the ring progressing clockwise until the node that owns the keyspace is found. This node will be the node which is closest but still is greater than the key. This node will be responsible for storage and lookup of this key. By generating the key using hash the keys are distributed evenly across the nodes in the network.

Overlay network

The overlay network is maintained by making each node maintain a set of links to its neighbors. The neighbor nodes are picked in a structural way that ensures that the hop number of a route and neighbor nodes per node is low. The set of links maintained by a node also has the property that for any key K, the node owns the K or has a link to a node that is closer to K.

By closer, we mean closer in the sense of distance the between node in the virtual overlay network. The distance metric may differ in the different DHT protocols. In Chord, the distance is defined as the distance between two keys on the circle. The distance traveling around the circle from a key k_1 to *a* key k_2 is $\delta(k_1, k_2)$. Kademlia uses another approach explained below.

Kademlia

Kademlia [10] is a commonly used overlay network protocol for decentralized peer to peer computer networks. It provides several useful features simultaneously unlike previous peer-to-peer system. In Kademlia the number of configuration messages between nodes for node detection is minimized. Key lookups spread this information automatically. The knowledge in a node is enough to make the nodes able to route queries through low latency paths. The queries are parallel and asynchronous. This to avoid timeout delays from failed nodes. The nodes in the network communicate by using the UDP protocol.

Each node in the network has its own ID. This 160 bit identifier is created when the node joins the system. The identifier is created in the same way as in Chord [see section 4.2.2.1.1]. Each message a node sends out contains the ID. This allows receiving nodes to record the sender's existence. This contact information is stored in a triple that contains IP address, UDP port and node ID. For each $0<i<160$ the node keeps a list for nodes of distance between 2^i and 2^{i+1} from itself. These lists are called k-buckets.

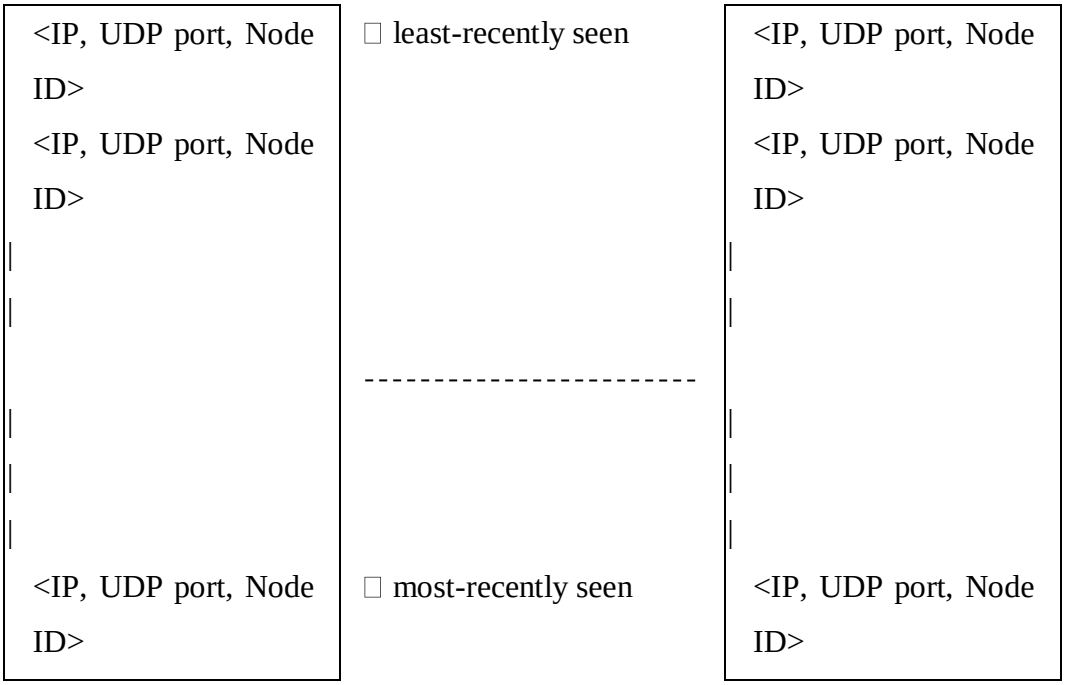




Figure 8: K-buckets

The keys stored in the nodes are also 160 bit identifier, for instance the SHA-1 hash of some data. The publishing and lookup of key, value pairs relies on a notion of distance between two identifiers. This is where Kademlia differs from Chord. Instead of using the circle distance, the distance between to identifiers is defined as the bitwise exclusive or (XOR) interpreted as an integer. The XOR is symmetric and allows nodes to receive lookup queries from the same distribution of nodes contained in their routing tables. This property makes the system learn useful routing information from the received queries. It also makes the routing more flexible compared to other peer-to-peer systems where routing is more rigid.

When a node receives a message (request or reply) from another node, the k-bucket corresponding to the sender's node ID is updated. K-buckets are implemented in a way that evicts least-recently seen nodes without removing live nodes. Studies have shown that the probability of a node being online in the future increases with the time a node has been online [26]. By keeping the oldest live contacts around, the probability that the nodes contained in the k-buckets will remain online is maximized [10]. The k-buckets also provide resistance to some types of DoS attacks. For instance, if new nodes are created rapidly in order to push out good nodes, it won't be noticeable.

Because of the constant traveling of data or requests through the nodes, buckets will stay fresh. In case of no traffic, a node refreshes its bucket by performing a node search for a random ID in its bucket. This is performed in hourly intervals if no traffic.

The protocol consist of four remote procedure calls (RPCs), PING, STORE, FIND_NODE and FIND_VALUE. PING is used for probing network nodes to check if it's online. STORE instructs a node to store a <key, value> pair in the nodes hash table. FIND_NODE and FIND_VALUE behave in the same way. Both take a160-bit ID as an argument and the recipient returns <IP address, UDP port, Node ID> triples for the k nodes it knows about closest to the target ID with one exception. If node receives a FIND_NODE RPC and previously has received a STORE RPC for the key, it only returns the stored value. The most important procedure of node in a Kademlia network performs is to locate the k closest nodes to a given ID. This procedure is known as node lookup and performed recursively.

5.4. Content Distribution Networks

What is a CDN?

A CDN (content delivery network), also called a *content distribution network*, is a group of geographically distributed and interconnected servers. They provide cached internet content from a network location closest to a user to speed up its delivery.

The primary goal of a CDN is to improve web performance by reducing the time needed to send content and rich media to users. Today, CDNs serve the bulk of web traffic, including traffic from well-known websites such as Facebook, Netflix and Amazon.

Why is using a CDN important?

A content delivery network is crucial for providers and website owners because it makes it easier to transfer content -- such as images, videos and website files -- to numerous servers spread out around the globe. This enhances user experience, especially for users who are located far from the origin server. It also speeds up web page loading times while reducing the strain on the main server.

As the content is served from numerous servers, a CDN can help increase the security and dependability of a website, making it less susceptible to security attacks and downtime.

How does a CDN work?

The process of accessing content cache on a CDN network edge is almost always transparent to the user. CDN management software dynamically calculates which server is located nearest to the user, making the request and delivering content based on those calculations. CDN edge servers communicate with the content's origin server to deliver to the user cached content and new content that hasn't been cached.

Edge servers reduce the distance that content travels and the number of hops a data packet makes. The result is less packet loss, optimized bandwidth and faster performance. This minimizes timeouts, latency and jitter, and it also improves the user experience. In the event of an internet attack or outage, fresh content cached and hosted on a CDN server remains available to those users who are nearest to the edge locations until the time-to-live for the CDN server expires. For example, if a website server goes down, the website is still accessible to visitors if it was cached properly on different geographically dispersed servers.

Organizations buy services from CDN providers for routing content to users from the nearest geographic location. CDN providers either host the content themselves or pay network operators and internet service providers to host CDN servers.

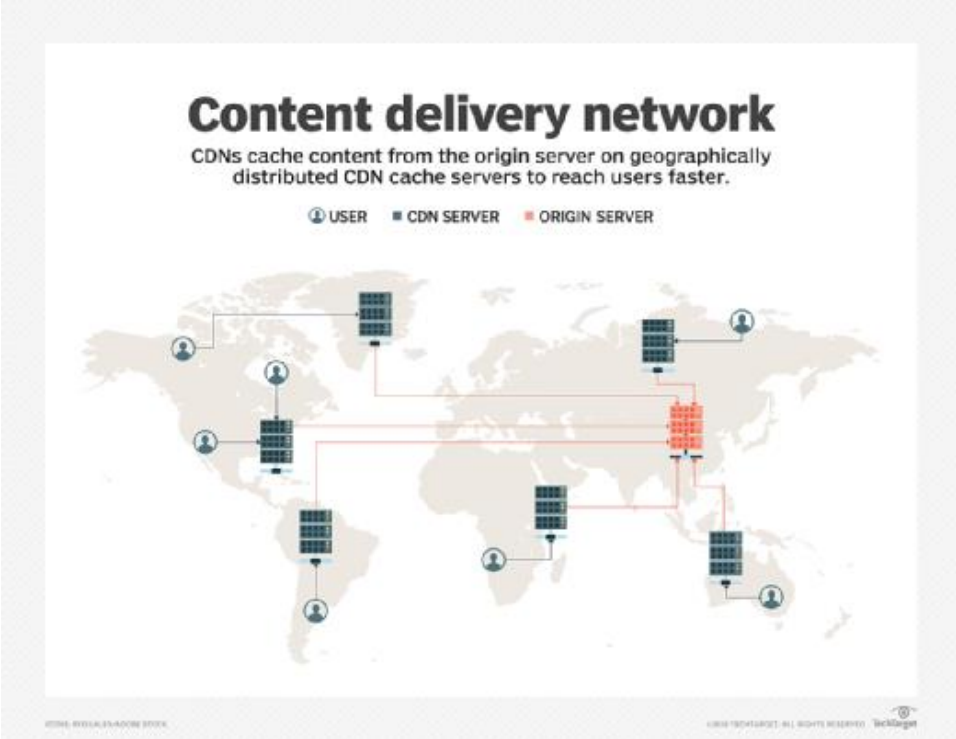
CDN providers also use load balancing and solid-state drives to help data reach users faster. They reduce file size using compression and special algorithms, and deploy machine learning and artificial intelligence to enable faster load and transmission times.

How does CDN caching work?

The functionality of a CDN is mostly dependent on caching static content. The data is stored and cached by the CDN edge servers.

The following is an example of a typical CDN caching process:

1. A web server in Chicago hosts static web content that a website visitor located in Washington, D.C., requests.
2. Once the origin server in Chicago receives the request, it sends a response to the website visitor. Simultaneously, it delivers a copy of the response to the CDN point-of-presence (POP) that's geographically closest to the website visitor.
3. The CDN POP stores its copy as a cached file.
4. When the same visitor or another visitor in that location makes a similar request again, the caching server -- not the origin server -- delivers the response.



CDN vs. web host

Despite using similar technology, CDNs and web hosts have distinct purposes. The following are the main differences between the two:

- Unlike web hosts, CDNs don't provide content across the last mile to consumers and only provide temporary content storage and caching.
- Web hosting usually provides access to a single server that handles everything from host name resolution to delivering requests. On the other hand, CDNs use a network of servers that are spread out globally, resulting in reduced response times for websites because the data is served from the closest edge location.
- A web host is responsible for hosting a website. Although a CDN doesn't host content, it caches it at the network edge to enhance performance. Most websites that struggle with performance issues opt for CDNs.
- Web hosts serve all types of content but CDNs only serve and host static data. The workload on CDN servers is lessened as a result, letting them concentrate on their main task of edge delivery.

- Issues, including restricted bandwidth, service outages and security risks, occasionally happen with traditional web hosting. CDNs address these problems by employing content caching on edge servers.

What are the uses of a CDN?

The main use of a CDN is to deliver content through a network of servers in a secure and efficient way. These networks improve the content security, performance and availability. Other CDN use cases include collecting user analytics and tracking data on network traffic. CDNs serve a range of content, including websites, mobile applications, streaming media and downloadable objects. Social media and search engine traffic go through CDNs.

Content-owning organizations use CDN services to accelerate the delivery of static and dynamic content, online gaming, mobile content and streaming video, as well as other specialized services.

Some examples of how CDNs are used include the following:

- An e-commerce organization uses a CDN to deliver content at busy times of the year when traffic spikes are likely.
- A bank uses a CDN to securely transfer sensitive customer data.
- A mobile application provider uses a CDN to reduce load times and increase response times, improving mobile users' customer experience.

What are the benefits of a CDN?

CDNs provide several advantages:

- **Efficiency.** CDNs improve web page load times and reduce bounce rates. Both advantages keep users from abandoning a slow-loading site or e-commerce application.
- **Security.** Besides content caching and web delivery, CDN providers offer services that complement their core functionality and capitalize on their presence at the

network edge. These services include distributed-denial-of-service (DDoS) attack mitigation, web application firewalls (WAFs) and bot mitigation.

- **Availability.** Since the website is distributed across multiple geographically dispersed servers, CDNs offer scalability. They can handle more traffic and avoid network failures better than origin servers. As a result, content is readily available.
- **Optimization.** These networks provide a diverse mix of performance and web content optimization services that complement cached site content.
- **Resource and cost savings.** CDNs reduce bandwidth consumption and costs.
- **Reduced latency.** CDN architecture is designed to reduce network latency caused by hauling traffic over long distances and across several networks. Eliminating latency is important as more dynamic content, video and software as a service are delivered to an increasing number of mobile devices. A reputable CDN provider can assist with delivering high-quality content at extremely low latency rates.
- **Caching.** CDN providers house cached content in either their POPs or third-party data centers. When a user requests content from a website that's also cached on a CDN, it redirects the request to the server nearest to the user and delivers the cached content from its location at the network edge. This process is invisible to the user.
- **Improved performance.** Many organizations use CDNs to cache website content to meet their performance needs. The demand for CDN services is increasing as websites offer more streaming video, e-commerce and cloud applications, where high performance is key. Few CDNs have POPs in every country. As a result, organizations must use several CDN providers to ensure they meet the needs of their customers and users, wherever they're located.
- **Special services.** Other services offered include web and application performance and acceleration services, streaming video and broadcast media optimization and digital rights management for video. Some CDN providers make their application programming interfaces (APIs) available to businesses to meet their unique needs.

What are examples of CDN platforms?

There are many CDNs available, each offering a variety of features. Products include the following:

- Akamai Technologies Inc. Intelligent Edge.
- Amazon CloudFront.
- ArvanCloud.
- CDN77.
- Cloudflare.
- Limelight Networks Edgio (formerly Yahoo Edgecast).

Some CDN providers, such as Cloudflare and Limelight, market their platforms as CDNs with added services, including DDoS or WAFs. Other providers, such as ArvanCloud, offer CDN services as part of several broader cloud services, such as cloud security and managed domain name system.

History of CDNs

CDNs are an integral part of modern internet architecture. They were born from a need to maintain efficient website performance and still serve that purpose today.

CDNs have evolved over the years and their evolution can be segmented into the following three generational types:

Static CDNs

Akamai launched the first static CDN in 1998. The company's techniques serve as the foundation of today's CDNs. First-generation CDNs focused mostly on static content, such as software downloads and audio and video streaming.

Static CDNs offered the following features:

- Served content included HTML and downloadable files.

- Used origin push caching.
- Improved server performance.
- Used a dispersed network topology.
- Cost a significant amount of money.
- Catered to the corporate sector.

Dynamic CDNs

As cloud and mobile computing gained traction, second-generation CDN services evolved. They enabled the efficient delivery of more complex dynamic multimedia and web content. As internet use grew, the number of CDN providers multiplied, along with the services they offered**.**

Dynamic CDNs offered the following features:

- Served both static and dynamic content, including rich media, to end users.
- Used origin pull caching.
- Used a consolidated network topology.
- Improve the efficiency and accessibility of servers and resources.
- Cost exorbitant amounts of money.
- Catered to the business sector.

Multi-purpose CDNs

The third-generation or next-generation CDN uses multiple CDNs from various CDN providers. A vast network of POPs is used instead of just one, which further improves the speed of content delivery, expands to wider regional and global coverages, and eliminates cybersecurity risks and outages.

Multi-purpose CDNs come with the following features:

- Serves both static and dynamic content, including rich media.
- Uses origin pull caching.
- Uses a highly consolidated network topology.
- Improves security, performance and availability.
- Costs an affordable amount of money.

The target audience for multi-purpose CDNs includes anyone who owns a website.

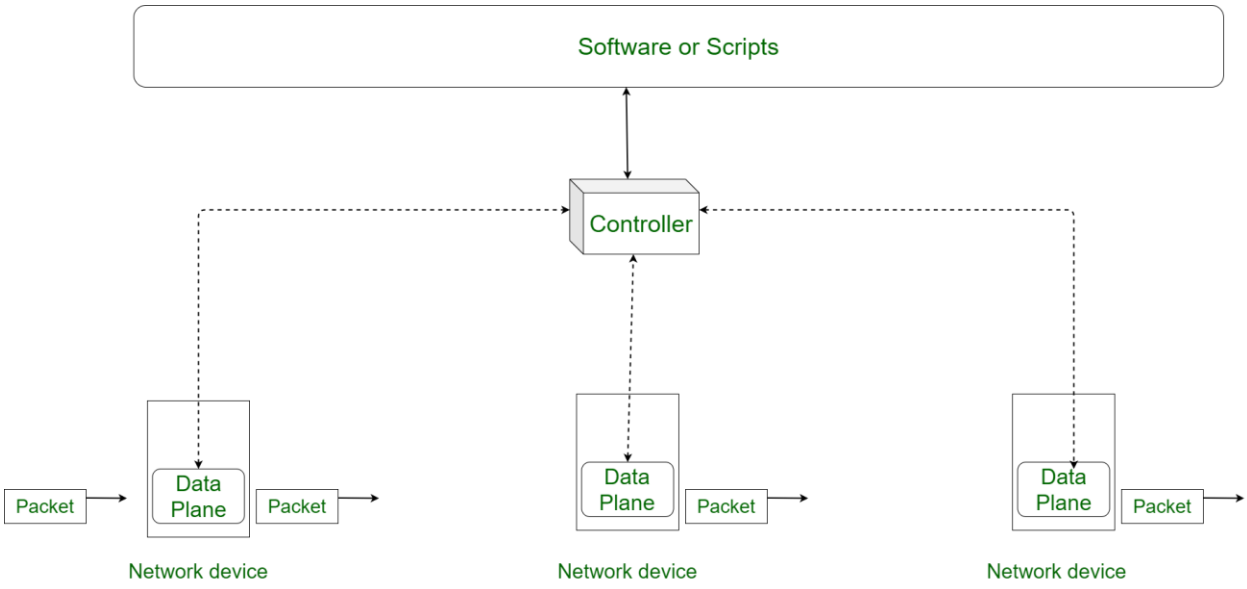
Some CDN business models are based on usage-based pricing or the volume of content delivered. Others charge a flat rate for basic services or even provide them for free, with additional fees for performance and optimization services.

5.5. Software Defined Networks: Architecture – Control and Data Planes

All traditional networking devices like router and switches uses distributed control plane. But newer model of networking i.e., **Software-defined Networking (SDN)** uses centralized control plane. Distributed control plane means that control plane of all networking devices lies within the device itself.

Each device have their own control plane to control data plane. In Centralized control plane system, there is a device which contains control plane of all devices. This device control the activities of data plane of all networking devices simultaneously. This device is called **Controller or SDN controller.**

The following figure shows a model of controller based networking.



1. **Southbound Interface :**

   In SDN, all networking devices must be connected to controller so that it can regulate data planes of all devices. When drawing architecture of network, usually the network architect places networking devices below controller. Now according to map conventions, interface between controller and networking devices lies to south of controller. Hence, these interfaces are called **Southbound Interface**.

Southbound interface is an interface between a program on controller and a program on networking device. Note that these interfaces we are discussing are software interface not physical one.

2. **Northbound Interface :**
   Controller need to know many information regarding network so that it can control data plane of networking devices All these information are provided by Network Programmer. Network Programmer provide essential information to controller through various software or script about what functions it has to do. Again these softwares/scripts are placed above controller in network architecture. This placement of software/script makes interfaces between controller and software in north direction, according to map conventions. Hence, Interfaces between controller and softwares are called **Northbound Interface**. These interfaces enable programmability of network.

All interfaces we discussed above are program based interfaces. These interfaces in a broader sense are called **Application Program Interface (API)**. An API is an interface through which two program can exchange data between them.

Let us understand- what is the importance of SDN and where SDN is used?

Preface

In the last two decades, the traffic demand and monitoring for networks have increased, since sales, customer service, internal communications, and documents are increasingly relying on network connectivity. Networking defined by software is a key technology needed to meet these new demands, coupled with the virtualization of network functions. SDN is just one piece of the jigsaw puzzle, and each offers network operators a new way of designing, deploying, and managing an SDN architecture and its service.

The basic principle of SDN is "to automate network supply, management and control through centralised policy/control engines based on software, that control network enabling endpoints".

Organizations can make innovation with SDN possible by creating in a record amount of time and with fewer complications than before new applications, services, and business models. With dynamic applications and a more efficient workflow, companies can create new services and paths to revenue. Network features can be executed with cheaper off-shelf hardware that reduces CAPEX levels. By supporting automation and algorithm control, companies can reduce the OPEX of IT services via the enhanced programming of network features, making it simple to design, deploy, manage and expand networks.

In short, it can be said that- SDN acts a "A Bigger Umbrella or a HUB" where rest other networking technologies come and sit under that umbrella and get merged with other platform to bring out the best of best outcome by decreasing the traffic rate and by increasing the efficiency of data flow.

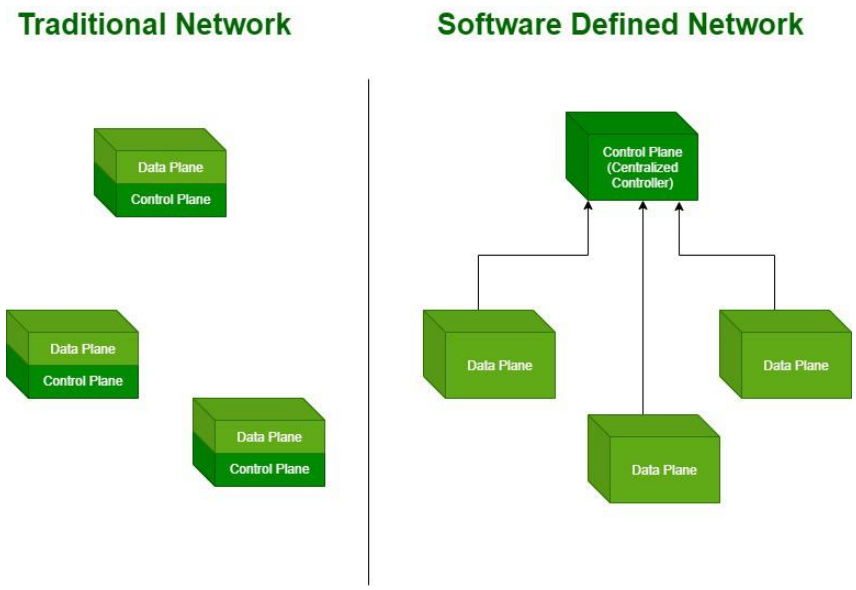
Where is SDN used?

Enterprises use SDN, the most widely used method for application deployment, to deploy applications faster while lowering overall deployment and operating costs. SDN allows IT administrators to manage and provision network services from a single location.

Cloud networking software-defined uses white-box systems. Cloud providers often use generic hardware so that the Cloud data center can be changed and the cost of CAPEX and OPEX saved.

Architecture of Software Defined Networks (SDN)

In traditional networks, the control and data plane are embedded together as a single unit. The control plane is responsible for maintaining the routing table of a switch which determines the best path to send the network packets and the data plane is responsible for forwarding the packets based on the instructions given by the control plane. Whereas in SDN, the control plane and data plane are separate entities, where the control plane acts as a central controller for many data planes.



There are many approaches that lead to the development of today's Software Defined Networks(SDN). They are:

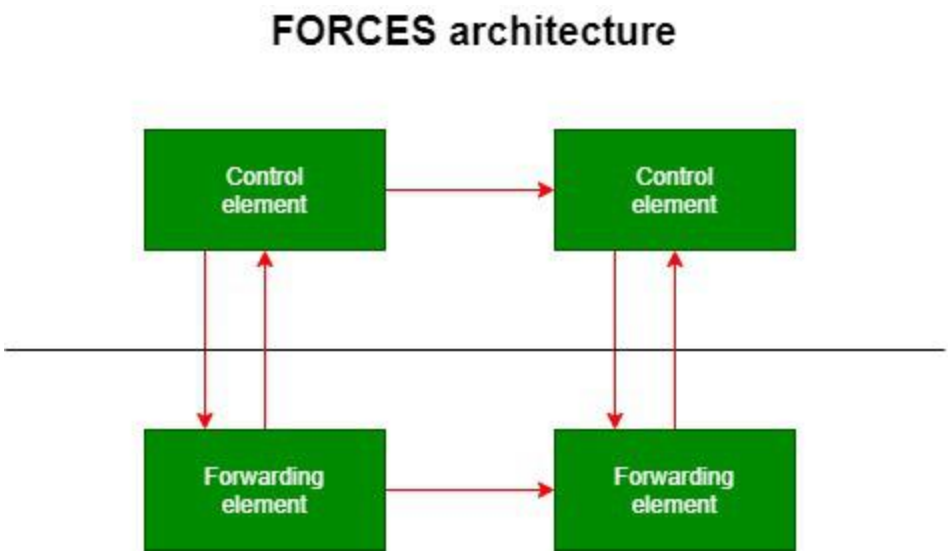
- Forces
- 4D approach
- Ethane

FORCES(Forwarding and control element separation):

The idea of separation of data plane(forwarding element) and control plane was first proposed by FORCES. It is said that hardware-based forwarding entities are controlled by a software-based control plane.

FORCES can be implemented in two ways:

1. The forwarding element and control plane are located within the same network device
2. The control element is taken off the device and placed in a separate system.



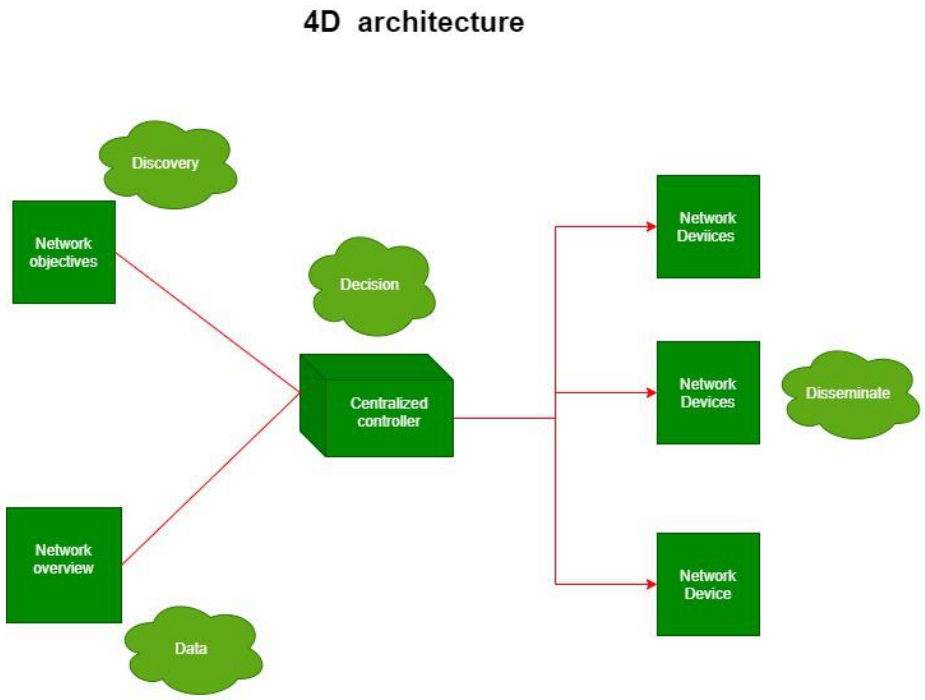
4D approach:

The 4D approach has four planes that control

- Decision
- Dissemination
- Discovery
- Data

It follows three principles:

- **Network-level objectives:** The objectives should be stated in terms of the whole network instead of individual devices. So that there won't be any need to depend on proprietary devices.
- **Network-wide view:** Decisions should be made based on the understanding of the whole network's traffic, topology, and events. Actions should be taken based on considering a network-wide view.
- **Direct control:** The control plane elements should directly be able to control the data plane elements. It should have the ability to program the forwarding table on individual devices.



Ethane: Ethane specifies network-level access of users which is defined by network administrators. Ethane is the exact forerunner of Software Defined Networks(SDN)

Principles of Ethane:

- High-level policies should inspect the network
- Routing should follow High-level policies.
- There should be a connection between packets and their origin in the network.

5.6. Open Flow, SDN Controllers

The basics of SDN and the OpenFlow Network Architecture

In recent years there has been a tremendous increase in carrier network traffic. To a great extent this has been fuelled by explosive growth in the use of online applications and cloud services by an ever-expanding array of wired and mobile connected devices. Network operators today need to cope with a vast array of data formats, service types and online devices, and of course they need to do so while ensuring availability, security, quality of service, all without increasing operating and equipment costs.

However, legacy network architectures and their management tools were not designed to cope with such highly elastic demand. This severely limits the operator's ability to cost-effectively respond to the scale, performance, and user experience requirements of today's dynamic environments, or to roll out differentiated services.

Software Defined Networking (SDN) has emerged as the industry's response to meeting these challenges. SDN allows networks to react dynamically to changes in usage patterns and availability of network resources. Network architectures can be instantly adjusted, respond to application and user requests, and services can be introduced far more quickly, easily and at a lower cost.

SDN provides separation between the control plane (controller) and data plane (switch) functions of networks using a protocol that modifies forwarding tables in network switches. This makes it possible to optimize networks on the fly and quickly respond to changes in network usage without the need for manually reconfiguring existing infrastructure or purchasing new hardware. SDN separates the control of network devices from the data they transport, and the switching software from the actual network hardware.

It also provides an entity, the controller**,** that has a comprehensive view of the entire network and its status, and with which switches (network resources) and applications (network consumers) can communicate in real-time. The controller makes it possible for networks to interact with applications and efficiently reconfigure themselves at need, allowing them to implement multiple logical network topologies on a single common network fabric.

OpenFlow. It is a multivendor standard defined by the Open Networking Foundation (ONF) for implementing SDN in networking equipment. The OpenFlow protocol defines the interface between an OpenFlow Controller and an OpenFlow switch**,** see Figure 1 below. The OpenFlow protocol allows the OpenFlow Controller to instruct the OpenFlow switch on how to handle incoming data packets.

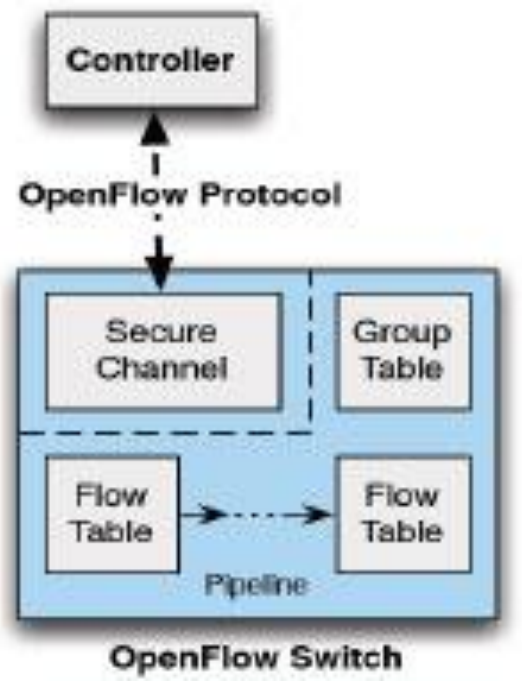


Figure 1 – The OpenFlow Protocol. Source: ONF OpenFlow 1.3.0 Switch Specification

The OpenFlow switch may be programmed to:
(1) identify and categorize packets from an ingress port based on a various packet header fields;
(2) Process the packets in various ways, including modifying the header; and,
(3) Drop or push the packets to a particular egress port or to the OpenFlow Controller.

The OpenFlow instructions transmitted from an OpenFlow Controller to an OpenFlow switch are structured as “flows”. Each individual flow contains packet match fields, flow priority, various counters, packet processing instructions, flow timeouts and a cookie. The flows are organized in tables. An incoming packet may be processed by flows in multiple “pipelined” tables before

exiting on an egress port. The OpenFlow protocol standard is evolving quickly with release 1.6 as the current revision at the time of this blog being published.

The OpenFlow Network Architecture consists of three layers:
(1) One or more OpenFlow virtual and/or physical switches;
(2) One or two OpenFlow controller(s); and,
(3) One or more OpenFlow application(s). For an illustration, see figure 2 below.

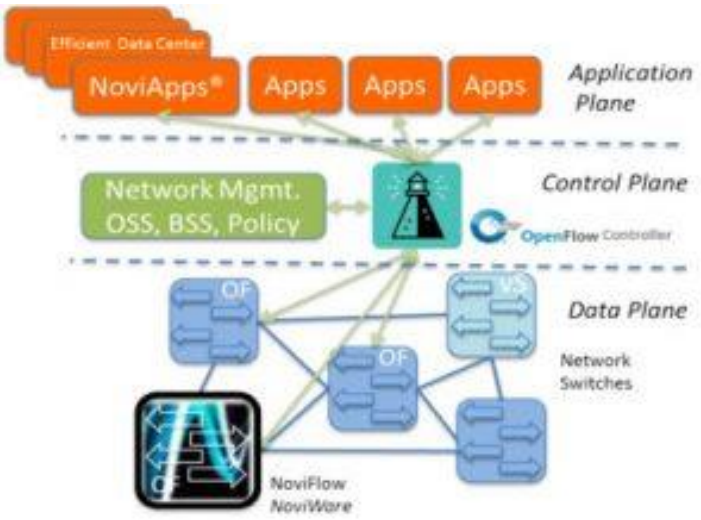


Figure 2 - The OpenFlow Architecture
Source: NoviFlow, Inc.

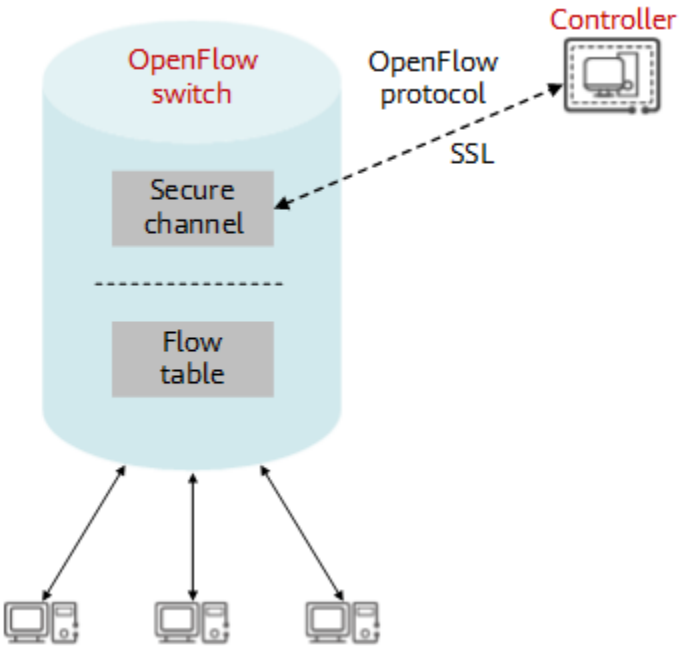
The OpenFlow controller maintains the OpenFlow protocol communications channels to the OpenFlow switches, maintains a local state graph of the OpenFlow switches and exposes a northbound API to the OpenFlow applications. The northbound API may be viewed as an abstraction of the network and allows the OpenFlow applications to read the state of the network and to instruct the network to perform various tasks.

A real world OpenFlow capable network may consist of only OpenFlow switches or a mixture of OpenFlow switches and traditional switches and routers. The latter network type is called an overlay network. Some OpenFlow applications will require only partial deployment of OpenFlow switches whereas others require a network consisting of only OpenFlow switches.

Most of the SDN network fabrics and applications we will discuss in this blog can be introduced as overlays, and multiple applications can be introduced in a staged fashion, with new ones building on the foundations laid by the previous applications.

How OpenFlow Works

The OpenFlow architecture consists of a controller, OpenFlow switch, and secure channel. The controller controls the network in a centralized manner to implement the functions of the control layer. The OpenFlow switch is responsible for forwarding at the data layer; it exchanges messages with the controller through a secure channel to receive forwarding entries and report its status.



OpenFlow architecture

OpenFlow Controller

An OpenFlow controller is the brain of the SDN architecture and is located at the control layer to instruct data forwarding through the OpenFlow protocol. Currently, mainstream OpenFlow controllers are classified into two types: open-source controllers and vendor-developed commercial controllers. The widely used open-source controllers include NOX, POX, and OpenDaylight. Huawei's iMaster NCE controllers are commercial ones.

OpenFlow Secure Channel

A secure channel is established between a controller and an OpenFlow switch. Through this channel, the controller controls and manages the switch, and receives feedback from the switch.

The messages exchanged over the OpenFlow secure channel must comply with the format specified by the OpenFlow protocol. The OpenFlow secure channel is usually encrypted using Transport Layer Security (TLS), but may be run directly over TCP in plain text in OpenFlow 1.1 and later versions. The following OpenFlow messages are transmitted over the channel:

- Controller-to-Switch message: is sent by the controller to the OpenFlow switch to manage or obtain the OpenFlow switch status.
- Asynchronous message: is sent by the OpenFlow switch to the controller to update network events or status changes to the controller.
- Symmetric message: is sent without solicitation by either the OpenFlow switch or the controller. It is mainly used to set up a connection and detect whether the peer is online.

OpenFlow Switch

As a core component of the OpenFlow network, an OpenFlow switch is mainly responsible for forwarding at the data layer. It can be a physical or virtualized switch/router. OpenFlow switches are classified into the following types based on their support for OpenFlow:

- Dedicated OpenFlow switch: is a standard OpenFlow device that supports only OpenFlow forwarding. The switch processes all traffic that passes through it in OpenFlow mode, and cannot perform Layer 2 or Layer 3 forwarding on the traffic.
- OpenFlow-compatible switch: supports both OpenFlow forwarding and Layer 2/3 forwarding. It is a commercial switch that supports OpenFlow features such as flow tables and secure channels.

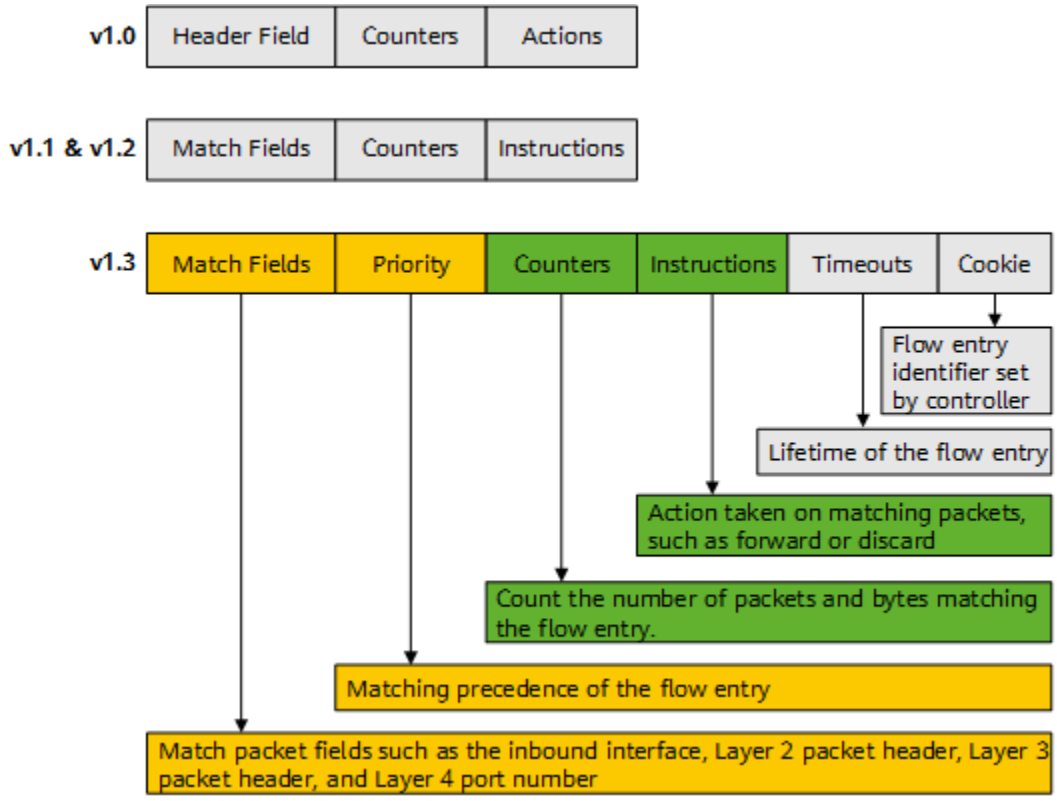
An OpenFlow switch forwards packets entering the switch based on the flow table, which contains a set of policy entries instructing the switch on how to process traffic. Flow entries are generated, maintained, and delivered by a controller.

Flow entry

Traditional network devices such as switches and routers forward data based on the locally saved Layer 2 MAC address forwarding table, Layer 3 IP address routing table, and transport-layer port numbers. OpenFlow switches forward data based on flow tables that contain network configuration information of all layers on the network, instead of 5-tuple information. The entries in a flow table are flexible combinations of certain keywords and actions.

Each flow entry in an OpenFlow flow table consists of match fields and a set of instructions applying to matching packets. When receiving a data packet, an OpenFlow switch parses and matches the packet header against match fields in the flow entries, and executes the corresponding instruction if a match is found.

The flow entry structure varies according to the OpenFlow versions, as shown in Figure 1-5.

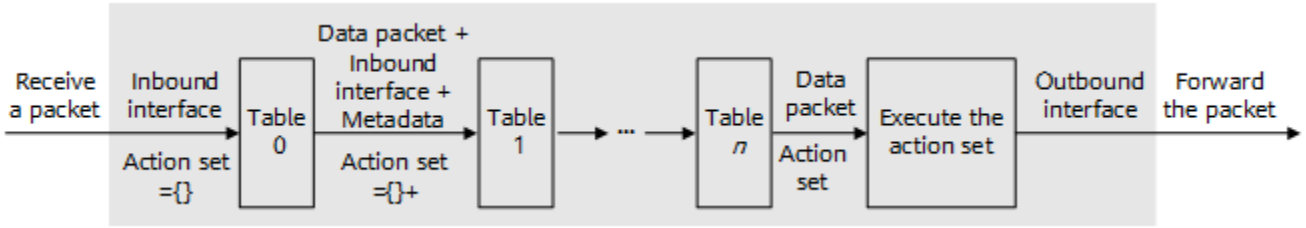


Flow entries in different OpenFlow versions

Multi-level flow table and pipeline processing

OpenFlow v1.0 uses a single flow table to match packets. This implementation is simple, but the flow table becomes quite large as various policies are configured to implement increasingly complex network requirements. This makes management of the control plane more difficult, and poses higher requirements on hardware.

OpenFlow v1.1 and later versions support multi-level flow tables and pipeline processing. When a packet arrives at a switch, matching starts from the flow table with the smallest sequence number, and may continue to additional flow tables in the pipeline. Multi-level flow tables can not only implement complex processing on data packets, but also reduce the length of a single flow table and therefore improve the entry lookup efficiency.



Multi-level flow table processing

Flow table delivery

OpenFlow flow tables can be delivered in proactive or reactive mode.

- In proactive mode, the controller delivers the flow table information to OpenFlow switches.
- In reactive mode, when an OpenFlow switch does not find any flow entry matching a received packet, it sends a message to the controller. The controller then determines how to forward the packet, and calculates and delivers the corresponding flow entry to the switch. In this mode, the switch does not need to maintain all flow entries. Instead, it obtains flow entries from the controller and saves them only for actually generated traffic, and deletes flow entries when they are aged out. Therefore, this mode greatly reduces the occupied chip space on the switch.

Application Scenarios of OpenFlow

With the development and popularization of OpenFlow, its research and application fields are also continuously expanded, covering network virtualization, security and access control, and load balancing. The following describes the typical application scenarios of OpenFlow:

- Application of OpenFlow on a college campus network

The network of scientific research institutions is where OpenFlow comes into being and also the network environment where OpenFlow is widely used. Students or researchers may need to verify newly designed network control protocols and data forwarding technologies when conducting innovative research on networks. In such cases, they want a platform separating the control plane from the forwarding plane, so that they can flexibly verify their research work. An OpenFlow-based network is such a platform meeting these requirements by providing almost the same complex environment as an actual network, while ensuring a good experiment effect and reducing experiment costs.

- Application of OpenFlow on a data center network

OpenFlow starts its large-scale application on cloud data center networks. Deployment of cloud data centers poses many virtualization requirements, such as dynamic creation of multi-tenant resources, traffic isolation, and dynamic migration of virtual machines (VMs). OpenFlow switches can work with the cloud management platform to implement dynamic allocation of network resources and on-demand transmission of network traffic, meeting network virtualization requirements of cloud services and improving network performance. As data centers have heavy traffic, data congestion may occur if transmission paths are not properly allocated, thereby hindering the operational efficiency of data centers. To avoid this situation, OpenFlow can be deployed in data centers to dynamically obtain traffic transmission information of each link and deliver OpenFlow flow entries to load balance traffic among links.

- Application of OpenFlow on an enterprise campus network

OpenFlow can be used on a campus network to effectively manage and control access devices, which are large in number, have a relatively high failure rate, but provide simple functions and traffic policies. With OpenFlow, a controller can be deployed to centrally deliver flow entries to access devices, and monitor and manage these devices. In scenarios where user identity

authentication is required, the authentication traffic can be diverted to the controller. After user identities are successfully authenticated, the controller delivers admission rules to the switch interfaces connected to users. If the controller detects that a network interface or traffic of a user is abnormal, it delivers rules to disable the interface or control the traffic to quickly rectify network faults, thereby improving network reliability and security.

5.7. Network Function Virtualization

The term “Network Functions Virtualization” (NFV) refers to the use of virtual machines in place of physical network appliances. There is a requirement for a hypervisor to operate networking software and procedures like load balancing and routing by virtual computers. A network functions virtualization standard was first proposed at the OpenFlow World Congress in 2012 by the European Telecommunications Standards Institute (ETSI), a group of service providers that includes AT&T, China Mobile, BT Group, Deutsche Telekom, and many more.

Need of NFV:

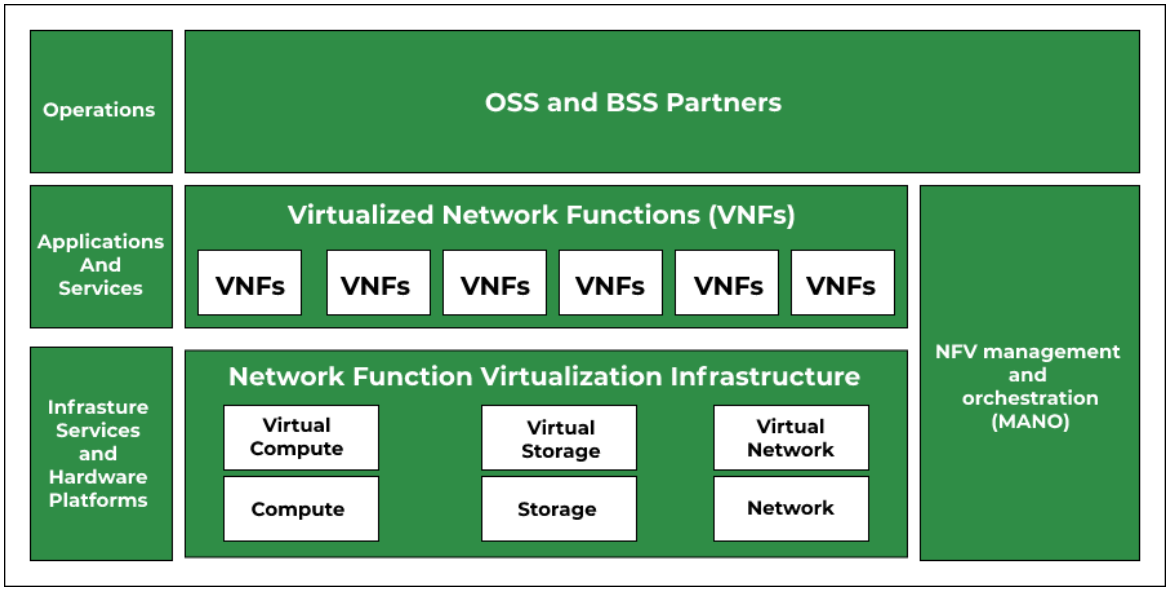
With the help of NFV, it becomes possible to separate communication services from specialized hardware like routers and firewalls. This eliminates the need for buying new hardware and network operations can offer new services on demand. With this, it is possible to deploy network components in a matter of hours as opposed to months as with conventional networking. Furthermore, the virtualized services can run on less expensive generic servers.

Advantages:

- Lower expenses as it follows Pay as you go which implies companies only pay for what they require.
- Less equipment as it works on virtual machines rather than actual machines which leads to fewer appliances, which lowers operating expenses as well.
- Scalability of network architecture is quite quick and simple using virtual functions in NFV. As a result, it does not call for the purchase of more hardware.

Working:

Usage of software by virtual machines enables to carry out the same networking tasks as conventional hardware. The software handles the task of load balancing, routing, and firewall security. Network engineers can automate the provisioning of the virtual network and program all of its various components using a hypervisor or software-defined networking controller.



Benefits of NFV:

- Many service providers believe that advantages outweigh the issues of NFV.
- Traditional hardware-based networks are time-consuming as these require network administrators to buy specialized hardware units, manually configure them, then join them to form a network. For this skilled or well-equipped worker is required.
- It costs less as it works under the management of a hypervisor, which is significantly less expensive than buying specialized hardware that serves the same purpose.
- Easy to configure and administer the network because of a virtualized network. As a result, network capabilities may be updated or added instantly.

Risks of NFV:

Security hazards do exist, though, and network functions virtualization security issues have shown to be a barrier to widespread adoption among telecom companies. The following are some dangers associated with implementing network function virtualization that service providers should take into account:

- **Physical security measures do not work:** Comparing virtualized network components to locked-down physical equipment in a data center enhances their susceptibility to new types of assaults.
- Malware is difficult to isolate and contain: Malware travels more easily among virtual components running on the same virtual computer than between hardware components that can be isolated or physically separated.
- **Network activity is less visible:** Because traditional traffic monitoring tools struggle to detect potentially malicious anomalies in network traffic going east-west between virtual machines, NFV necessitates more fine-grained security solutions.

NFV Architecture:

An individual proprietary hardware component, such as a router, switch, gateway, firewall, load balancer, or intrusion detection system, performs a specific networking function in a typical network architecture. A virtualized network substitutes software programs that operate on virtual machines for these pieces of hardware to carry out networking operations.

Three components make up an NFV architecture:

- **Centralized virtual network infrastructure:** The foundation of an NFV infrastructure can be either a platform for managing containers or a hypervisor that abstracts the resources for computation, storage, and networking.
- **Applications:** Software delivers many forms of network functionality by substituting for the hardware elements of a conventional network design (virtualized network functions).
- **Framework:** To manage the infrastructure and provide network functionality, a framework is required (commonly abbreviated as MANO, meaning Management, Automation, and Network Orchestration).

Why network functions virtualization?

NFV allows for the separation of communication services from dedicated hardware, such as routers and firewalls. This separation means network operations can provide new services

dynamically and without installing new hardware. Deploying network components with network functions virtualization takes hours instead of months like with traditional networking. Also, the virtualized services can run on less expensive, generic servers instead of proprietary hardware.

Additional reasons to use network functions virtualization include:

- **Pay-as-you-go**: Pay-as-you-go NFV models can reduce costs because businesses pay only for what they need.
- **Fewer appliances**: Because NFV runs on virtual machines instead of physical machines, fewer appliances are necessary and operational costs are lower.
- **Scalability**: Scaling the network architecture with virtual machines is faster and easier, and it does not require purchasing additional hardware.

How does network functions virtualization work?

Essentially, network functions virtualization replaces the functionality provided by individual hardware networking components. This means that virtual machines run software that accomplishes the same networking functions as the traditional hardware. Load balancing, routing and firewall security are all performed by software instead of hardware components. A hypervisor or software-defined networking controller allows network engineers to program all of the different segments of the virtual network, and even automate the provisioning of the network. IT managers can configure various aspects of the network functionality through one pane of glass, in minutes.

Benefits of network functions virtualization

Many service providers feel that the benefits of network functions virtualization outweigh the risks. With traditional hardware-based networks, network managers have to purchase dedicated hardware devices and manually configure and connect them to build a network. This is time-consuming and requires specialized networking expertise.

NFV allows virtual network function to run on a standard generic server, controlled by a hypervisor, which is far less expensive than purchasing proprietary hardware devices. Network configuration and management is much simpler with a virtualized network. Best of all, network

functionality can be changed or added on demand because the network runs on virtual machines that are easily provisioned and managed.

Risks of network functions virtualization

NFV makes a network more responsive and flexible, and easily scalable. It can accelerate time to market and significantly reduce equipment costs. However, there are security risks, and network functions virtualization security concerns have proven to be a hurdle for wide adoption among telecommunications providers. Here are some of the risks of implementing network functions virtualization that service providers need to consider:

- **Physical security controls are not effective**: Virtualizing network components increases their vulnerability to new kinds of attacks compared to physical equipment that is locked in a data center.
- **Malware is difficult to isolate and contain**: It is easier for malware to travel among virtual components that are all running off of one virtual machine than between hardware components that can be isolated or physically separated.
- **Network traffic is less transparent**: Traditional traffic monitoring tools have a hard time spotting potentially malicious anomalies within network traffic that is traveling east-west between virtual machines, so NFV requires more fine-grained security solutions.
- **Complex layers require multiple forms of security**: Network functions virtualization environments are inherently complex, with multiple layers that are hard to secure with blanket security policies.

NFV vs. SDN

- While NFV separates networking services from dedicated hardware appliances, software-defined networking, or SDN, separates the network control functions such as routing, policy definition and applications from network forwarding functions. With SDN, a virtual network control plane decides where to send traffic, enabling entire networks to be programmed through one pane of glass. SDN allows network control functions to be automated, which makes it possible for the network to respond quickly to dynamic workloads. A software-

defined network can sit on top of either a virtual network or a physical network, but a virtual network does not require SDN to operate. Both SDN and NFV rely on virtualization technology to function.